

EasyOutPainter: One Step Image Outpainting with both Continuous Multiple and Resolution

Shaofeng Zhang¹, Qiang Zhou², Zhibin Wang², Hao Li³, Junchi Yan^{1†}

{sherrylone, yanjunchi}@sjtu.edu.cn, {zhouqiang, zhibin.waz}@inftech.ai, lihao_lh@fudan.edu.cn

¹School of Artificial Intelligence and School of Computer Science, Shanghai Jiao Tong University

²INF Tech Co., Ltd., ³Fudan University

Abstract—Image outpainting aims to generate the content of an input sub-image outside its boundaries, which remains open for existing generative models. This paper explores image outpainting in three directions that have not been achieved in literature to our knowledge: outpainting 1) with continuous multiples (in contrast to the discrete ones by existing methods); 2) with arbitrary resolutions; and 3) in a single step (for any multiples and resolutions). The arbitrary multiple outpainting is achieved by utilizing randomly cropped views from the same image during training to capture arbitrary relative positional information. Specifically, by feeding one view and relative positional embeddings as queries, we can reconstruct another view. At inference, we generate images with arbitrary expansion multiples by inputting an anchor image and its corresponding positional embeddings. The continuous-resolution outpainting is achieved by introducing the multi-scale training strategy into generative models. Specifically, by disentangling the image resolution and the number of patches, it can generate images with arbitrary resolutions without post-processing. Meanwhile, we propose a query-based contrastive objective to make our method not rely on a pre-trained backbone network which is otherwise often required in peer methods. The comprehensive experimental results on public benchmarks show its superior performance over state-of-the-art approaches.

Index Terms—Image Outpainting, Generative Models, Diffusion Models, Contrastive Learning.

I. INTRODUCTION

Image outpainting [1]–[3], a.k.a. image extrapolation [4]–[6] or view expansion [7], which can be regarded as a special case for image completion. It refers to generate new content beyond the original boundaries of a given sub-image. With the fast progress in image generation technologies and applications, image outpaint attracts increasing attention from both academy and industry in recent years, for its technical challenges as well as practical value ranging from camera view recommendation [8] to video stabilization [9] etc. Compared with its more matured counterpart i.e. image inpainting [10] that tries to fill the centering content given the rich surrounding areas, outpainting is a more emerging task which remains relatively open, compared with other condition-based generation settings like style transfer [11] etc. In fact, image outpainting can find wide applications (e.g., scene graph completion [12], unbounded recommendation of camera view [8] and image composition [4]). In general, an ideal outpainter is expected to achieve the following basic functions [13]: 1) determining

where the missing regions should be located relative to the output’s spatial locations for both nearby and faraway features; 2) guaranteeing that the extrapolated image has a consistent structural layout with the given sub-image; and 3) the borders between extrapolated regions and original input images should be visually smooth. Moreover, a flexible approach is welcomed in practice in terms of its efficiency and flexibility.

These requirements have been mainly addressed by existing outpainting methods which generally adopt GAN-based frameworks [7], [16], whereby random noise and the initial input sub-images (as conditions) are used to generate the fake surrounding image content, and the discriminator is adopted to classify the generated images as fake or real. A recent method [13] employs the (Masked AutoEncoder) MAE-based framework [19]. Specifically, this method as called QueryOTR, models the extrapolation as the MIM (Masked Image Modeling) problem [23] by replacing the extrapolated regions around the input sub-images with masked tokens and predicting the pixels of the masked patches. Although these methods can generate realistic images, they still suffer from three applicability limitations:

1) As shown in Fig. 1(a), they require running multiple times to outpaint the image (e.g., the SOTA method in [13] outpaints 11.7x images by passing through the model three times ($x \rightarrow \text{forward} \rightarrow 2.25x \rightarrow \text{forward} \rightarrow 5x \rightarrow \text{forward} \rightarrow 11.7x$), which is inefficient especially when the required expansion multiple is large (e.g., 99x);

2) As described in Table I, they can only generate images with fixed resolutions;

3) The discriminator-based architecture can slow down the convergence [14], [24], and requires a pre-trained encoder. In other words, there are pretraining costs in their model, which are usually very high (up to 1,000 or longer epochs on ImageNet).

In this paper, we propose EasyOutPainter as an efficient and flexible solution to address the above three limitations. It can be easily integrated into any generative frameworks (e.g., GAN [14] and diffusion [21]). We make clear comparisons in Table I to better position our method and its preliminary conference version [22]. The main contribution can be summa-

†: Correspondence author. This work was partially supported by National Natural Science Foundation of China (62222607) and Shanghai Municipal Science and Technology Major Project (2021SHZDZX0102).

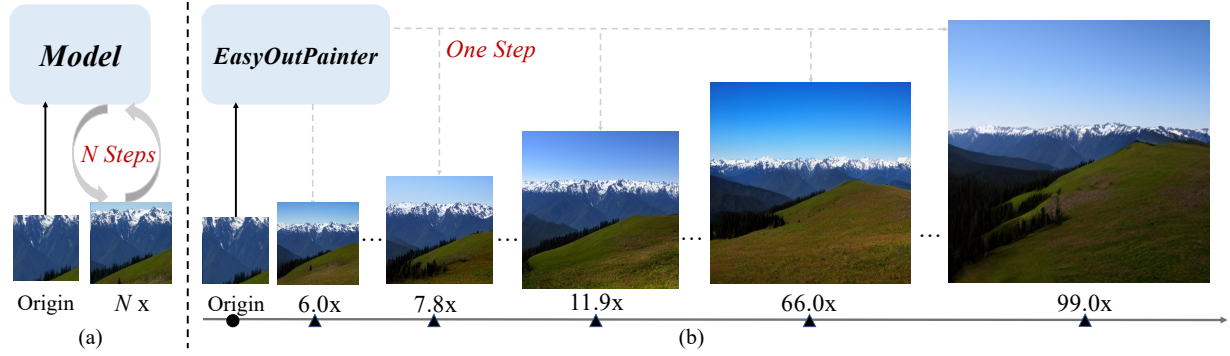


Fig. 1. EasyOutPainter can outpaint images with arbitrary multiples and resolutions in one step (b). In contrast, previous methods (a) outpaint images with discrete multiples in multiple steps. Note that $N \times$ here means an N -times larger image needs to be generated, while 2.25x, 5x, and 11.7x are adopted in the experiment following the setting of the previous work [13] for fair comparisons.

TABLE I
METHODOLOGY COMPARISONS OF THE PROPOSED EASYOUTPAINTER WITH RECENT ADVANCED IMAGE OUTPAINTING METHODS. PIXEL LOSS MEANS PIXEL-WISE l_1 OR l_2 LOSS ON THE GENERATED IMAGES. ‘FEATURE LOSS’ MEANS l_2 LOSS ON THE FEATURE MAP; ‘NO NEED PRETRAIN’ MEANS THE BACKBONE IS RANDOMLY INITIALIZED WITHOUT THE NEED OF EXTRA PRETRAINING COST.

Method	Generation type	Objectives	Encoder	no need pretrain	Continuous multiple	One-step	Arbitrary resolution
NSIPO [7]	GAN-based [14]	Pixel+GAN loss	ResNet-50 [15]	✓	✗	✗	✗
IOH [16]	GAN-based [14]	Pixel+GAN loss	Conv layers	✓	✗	✗	✗
Uformer [17]	GAN-based [14]	Pixel+GAN+Feature	Swin Trans [18]	✗	✗	✗	✗
QueryOTR [17]	GAN [14]+MAE [19]	Pixel+GAN Loss	ViT [20]	✗	✗	✗	✗
Vanilla Diff [21]	Diffusion-based	Pixel Loss	ViT [20]	✓	✗	✗	✗
PQDiff [22] (conference version)	Diffusion-based	Pixel Loss	ViT [20]	✓	✓	✓	✗
EasyOutPainter	Diffusion-based	Pixel Loss	ViT [20]	✓	✓	✓	✓

alized as follows¹, and the source code will be made publicly available at <https://github.com/Sherrylone/EasyOutPainter>.

i) Continuous multiples for image outpainting. We propose EasyOutPainter, which learns the positional relationships and pixel information simultaneously. Specifically, in the training stage, EasyOutPainter first randomly crops the given images twice to generate two views (called the anchor view and target view). Then, it learns to predict the target view from the anchor view and the pre-calculated relative positional

embeddings (RPE) of the two views. Since the RPE can represent continuous relationships between two views, it can outpaint the images in continuous multiples. Besides, since we can adjust the RPE in the inference stage, it can outpaint images in only one step. To our best knowledge, we are the first to outpaint images in continuous multiples (e.g., 6.0x, 7.8x, 66.0x, 99.0x), whereas the SOTA QueryOTR [13] can only outpaint images in discrete multiples. Besides, we are also the first to achieve outpainting in one step, whereas [7], [13] can only outpaint images step-by-step, which severely limits their sampling, i.e. generation efficiency. Specifically, under the 2.25x, 5x, and 11.7x outpainting settings, our EasyOutPainter only takes 46.0%, 23.0%, and 15.4% of the time of the SOTA QueryOTR [13].

ii) Arbitrary resolutions for outpainting. We propose a multi-scale training strategy, which enables to generate images with arbitrary resolutions without post-processing modules (e.g., super-resolution). Specifically, we interpolate the target view with different resolutions. Then, we propose a new interpolating function of the relative positional embeddings to keep the same size of the target view, which enables our approach to predict the target view with flexible size.

iii) Model architecture and contrastive acceleration. We propose a position-aware cross-attention mechanism between relative positional embedding and input sub-image patches, which helps EasyOutPainter to disentangle the resolution of anchor views and target views, enabling to outpaint images with continuous multiple and arbitrary resolutions. Besides,

¹Part of the work has been published in the preliminary conference version: PQDiff [22], which introduces two-crop paradigm and relative positional embeddings to achieve continuous-multiple outpainting in one step (Note that “one step” here differs from a diffusion timestep. Here, “one step” means that for any outpainting factor, we only need to input the image into EasyOutPainter “once”). This journal version aims to extend the fixed-resolution outpainting to continuous-resolution outpainting. The main improvements include: 1) propose the multi-scale training strategy and multi-scale relative positional embedding, which enable the proposed EasyOutPainter can outpaint images with continuous resolutions which was not achieved in the conference version [22] neither in any other literature; 2) propose a contrastive loss (take the anchor and target views from the same image as positive pairs) behind the cross-attention layers, which greatly improves the convergence speed e.g., with the same 30k iterations, it improves the FID score by 50% (the lower the better) from 149.347 to 72.404 (see Fig. 9), and achieves the final score of FID=2.311 and IS=0.435 on the Scenery [7] dataset; 3) introduce image-conditioned classifier-free guidance on the diffusion trajectory, which balances the quality and relevance of the input conditions (As shown in Table II, EasyOutPainter improve previous SOTA 27.111 (60.911 to 33.866), 28.964 (64.926 to 35.962), and 42.274 (69.951 to 27.677) FID scores on the Scenery, Buildings, and Wikiart datasets under 11.7x large outpainting multiple setting, respectively); 4) extensive additional (ablation) experiments including classifier-free guidance, convergence speed, multi-scale sampling, and sampling efficiency; 5) the paper is rewritten with comprehensive discussions and more details.

we propose the query-based contrastive objectives to further accelerate the convergence speed, which helps to achieve SOTA results without any pre-trained encoders.

iv) New SOTA performance especially without pre-training. Experimental results on outpainting benchmarks [7], [17] show that EasyOutPainter significantly surpasses QueryOTR [13] and achieves new SOTA (58.629 to **33.866**, 64.926 to **35.962** and 69.951 to **27.677** FID scores with the challenging 11.7x multiple setting on the Scenery, Building Facades, and WikiArts datasets, respectively). Moreover, it achieves new FID SOTA results across settings (2.25x, 5x, and 11.7x).

II. BACKGROUND AND RELATED WORK

In this section, we review the literature related to image outpainting and diffusion models, as well as contrastive learning, which are the main techniques used in our approach.

A. Image Outpainting

It aims to generate the surrounding regions from the visual content, which can be considered as an image-conditioned generation task [25]–[30]. The work [31] brings the image outpainting task to attention with a deep neural network inspired by image inpainting [10]. It focuses on enhancing the quality of generated images smoothly by using GANs and post-processing to perform horizontal outpainting. The work [16] designs a CNN-based encoder-to-decoder framework by using GAN for image outpainting. In [32], a Semantic Regeneration Network is proposed to directly learn the semantic features from the conditional sub-image. Meanwhile, a 3-stage model is developed in [33] with an edge-guided generative network to produce semantically consistent output. Although these methods avoid bias in the general padding and up-sampling pattern, they still suffer from blunt structures and abrupt color issues, which tend to ignore spatial and semantic consistency. To tackle these issues, a Recurrent Content Transfer (RCT) block is devised [7] for temporal content prediction with Long Short Term Memory (LSTM) networks [34]. To enrich the context, [35] additionally switches the outer area of images into its inner area. QueryOTR [13] further proposes the PSM (patch smoothing module) and QEM (query expansion module), and integrates them into MAE [19] framework to perform outpainting. On the basis of QueryOTR [13], UNIC [8] proposes two new applications of image outpainting: the unbounded recommendation of camera views and feature extrapolation, where the feature extrapolation is commonly used in variant predictive tasks.

We particularly discuss QueryOTR [13], for its so-far best performance to our best knowledge in literature, especially its adoption of the ViT module [20], which also will be used in our approach. Specifically, QueryOTR proposes to adopt the ViT-based encoder and MIM-based [23] architecture for outpainting. Specifically, given an input sub-image $\mathbf{x} \in \mathbb{R}^{H \times W \times 3}$, QueryOTR first partitions $\mathbf{x}$ into regular non-overlapping patches with the patch size $P \times P$ to obtain the patch tokens $\{\mathbf{x}_p^1, \mathbf{x}_p^2, \dots, \mathbf{x}_p^L\}$, where P is usually set as 16 and $L = \frac{H \times W}{P^2}$. The goal of QueryOTR is to predict the extra sequence $\{\mathbf{x}_p^{L+1}, \mathbf{x}_p^{L+2}, \dots, \mathbf{x}_p^{L+R}\}$ representing the

extrapolated regions. In line with MAE [19], the sin-cos positional embedding of input patches is pre-defined. Thus, the positional embedding of the extrapolated patches can be obtained. The training of QueryOTR also follows MAE [19], where the visible patches sequence $\{\mathbf{x}_p^1, \mathbf{x}_p^2, \dots, \mathbf{x}_p^L\}$ is fed to the encoder. Then, the fixed positional embedding of the masked tokens sequences $\{\mathbf{x}_p^{L+1}, \mathbf{x}_p^{L+2}, \dots, \mathbf{x}_p^{L+R}\}$ is used as input to the decoder to predict the extrapolated patches. Finally, QueryOTR copies the input sub-image to the generated image in the corresponding position, followed by the Patch Smoothing Module (PSM) to smooth the border.

B. Diffusion Models

These models (e.g., the seminal work [21]) gradually inject noise into data and then reverse this process to generate data from noise. The noise-injection process is also called the forward process. Given the original data $\mathbf{x}_0$ (clean), then, the forward process can be formalized as a Markov chain: $q(\mathbf{x}_{1:T}|\mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1})$ where q is the forward process and $q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t|\sqrt{\alpha_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I})$, and α and β represent the noise schedule and $\alpha + \beta = 1$. $\mathcal{N}(0, 1)$ means the standard Gaussian noise. To reverse this process, a Gaussian model $p(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}|\mu_t(\mathbf{x}_t), \sigma_t^2\mathbf{I})$ is adopted to approximate the ground truth reverse transition $q_{\mathbf{x}_{t-1}|\mathbf{x}_t}$. Specifically, the optimal mean value of $\mathbf{x}_t$ can be written as [36]: $\mu_t^*(\mathbf{x}_t) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\alpha_t}} \mathbb{E}[\epsilon|\mathbf{x}_t] \right)$ where $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$, and ϵ is the standard Gaussian noises injected to $\mathbf{x}_t$. Thus, the learning is equivalent to a noise prediction task. Formally, a noise prediction network $\epsilon_\theta(\mathbf{x}_t, t)$ is used to learn $\mathbb{E}[\epsilon|\mathbf{x}_t]$ by minimizing the noise prediction objective. For l_2 loss, we can formulate the objective of noise prediction task as $\min_\theta \mathbb{E}_{t, \mathbf{x}_0, \epsilon} \|\epsilon - \epsilon_\theta(\mathbf{x}_t, t)\|_2^2$, where t is uniform between 1 and T . On the basis of the plain diffusion models, LDM [37] proposes to add noise and denoise in the latent space, which greatly improves the training efficiency. Followed by LDM, ViTDiff [38] proposes to replace CNN-base U-net [39] with ViTs [20] to estimate the backward process in diffusion models. Except for image generation, diffusion models have been also widely used in many other applications, e.g., object detection [40], video generation [41], segmentation [42].

C. Contrastive Learning

Contrastive learning aims at learning the global information of images through enforcing contrastive objectives (e.g., InfoNCE loss [43]) on the final global image representations ($[CLS]$ token or from average pooling). The pivotal idea is to align the embeddings of two augmented views from the same image while preventing trivial solutions (a.k.a. degenerated representations). To reach this target, MoCo [44], [45] series employ a memory bank to store and update negative examples, from which the negative examples could be randomly sampled, whereas SimCLR [46] treats all other data samples within the same training batch as negative examples. Due to the inconvenience and huge cost of using negative examples, later research turned to explore negative-example-free methods. BYOL [47] and SimSiam [48] design asymmetric architectures with additional predictors and stop-gradient to prevent

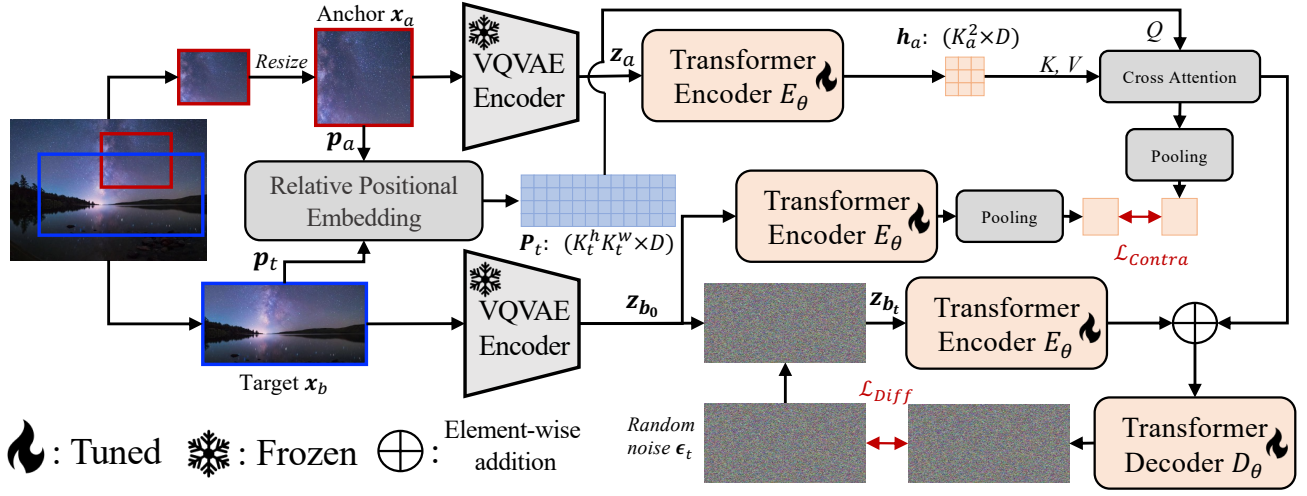


Fig. 2. Training pipeline of the proposed EasyOutPainter. Given an input image, we first randomly crop the image twice to generate two views (called the anchor view $\mathbf{x}_a$ and target view $\mathbf{x}_b$). Then, we resize the anchor view to $H_a \times W_a$, (defaultly, $H_a = W_a$) and compute the relative positional embedding $\mathbf{P}_b \in \mathbb{R}^{K_b^H K_b^W \times D}$. Then, we feed the two views into the fixed VQVAE encoder to generate clean data $\mathbf{z}_a$ and $\mathbf{z}_b$. For the **contrastive** branch, we input the anchor embedding to the transformer encoder, followed by a cross-attention module and global pooling module to calculate $\mathbf{h}_{b_0}$. Meanwhile, the **clean** data $\mathbf{z}_{b_0}$ is also fed to the transformer encoder to calculate the embedding of the target view $\mathbf{h}_{b_0}$. Then, the loss is added on the two latent embeddings $\mathbf{h}_{b_0}$ and $\mathbf{h}_{b_0}$. For the **diffusion** branch, we add the random Gaussian noise ϵ_t on $\mathbf{z}_{b_0}$ to generate noisy input $\mathbf{z}_{b_t}$. Then, we input the $\mathbf{z}_{b_t}$ to the transformer encoder to calculate $\mathbf{h}_{b_t}$ and add $\mathbf{h}_{b_t}$, followed by the transformer decoder to predict the noise $\tilde{\epsilon}_t$.

using negative examples. Barlow Twins [49], ZeroCL [50], ARB [51], and VICReg [52] resort to feature-level decorrelation to mitigate trivial solutions. Inspired by the success of Vision Transformers (ViT), CNN backbones are gradually replaced with ViTs [53], [54]. iBOT [55] further incorporates contrastive learning with masked image modeling and has achieved State-of-the-Art performance on various tasks.

III. THE PROPOSED EASYOUTPAINTER

We provide an embodiment based on the diffusion model and we term it as EasyOutPainter, whereby the continuous multiples, arbitrary resolutions, and one-step generation are achieved. Our model with these advantages can incorporate standard generative models *e.g.*, GANs (and the empirical performance comparison given in our ablation studies), and diffusion. We also show the significant improvements compared with a vanilla diffusion model directly for outpainting.

Overview. Our approach consists of four key modules: view generation and relative positional embedding, diffusion process, cross attention in the position-aware transformer model, and denoising and contrastive objectives. The relative positional embedding is fed to the cross-attention module, where the cross-attention module is injected into the U-ViT [38] $\mathbf{U}_\theta$, and used for the backward diffusion process (details can be seen in Fig. 2).

A. View Generation and Relative Positional Embedding

Given the image $\mathbf{x} \in \mathbb{R}^{H \times W \times 3}$ from the training set, we first randomly crop the image twice (the resolutions of the cropped view are the multiples of $r_{vq} \cdot \text{PatchSize}$, where r_{vq} is the compression ratio of VQVAE and PatchSize means the patch size of the transformer encoder) to generate the anchor

view $\mathbf{x}_a$ and the target view $\mathbf{x}_b$. Then, we resize the anchor view to $H_a \times W_a \times 3$, while the target view $\mathbf{x}_b$ keeps its original resolutions ($H_b \times W_b \times 3$). Note that we randomly crop two views, and as a result, the positional relationship of the two views might be either containing, overlapping, or non-overlapping. Meanwhile, we can record the absolute position $\mathbf{p}_a$ of $\mathbf{x}_a$ and $\mathbf{p}_b$ of $\mathbf{x}_b$, respectively. Specifically, $\mathbf{p}_a$ and $\mathbf{p}_b$ are composed of $\mathbf{p}_a = \{p_{ai}, p_{aj}, p_{ah}, p_{aw}\}$ (top location, left location, height and width) and $\mathbf{p}_b = \{p_{bi}, p_{bj}, p_{bh}, p_{bw}\}$, respectively. Then, we feed the anchor view and target view to the VQVAE encoder, which can be expressed as:

$$\mathbf{z}_a = E_{VQVAE}(\mathbf{x}_a), \quad \mathbf{z}_b = E_{VQVAE}(\mathbf{x}_b) \quad (1)$$

where E_{VQVAE} means the encoder of VQVAE [56]. Consequently, the size of the anchor view becomes $\frac{H_a}{r_{vq}} \times \frac{W_a}{r_{vq}} \times d_{vq}$, where d_{vq} is the hidden dimension of VQVAE. Then, since we can early access the architecture of the transformer encoder, we know the patch size, and we denote the patch size as PatchSize . For simplicity, we denote:

$$K_{a(b)}^{H(W)} = \frac{H(W)_{a(b)}}{r_{vq} \cdot \text{PatchSize}} \quad (2)$$

By setting each patch size of the anchor view as 1 (note that 1 is for simple representation, we can also set as other numbers), the positions of the **anchor** view and be formulated as:

$$\mathbf{rp}_a = \begin{pmatrix} (0, 0) & (0, 1) & (0, 2) & \cdots & (0, K_a^W) \\ (1, 0) & (1, 1) & (1, 2) & \cdots & (1, K_a^W) \\ (2, 0) & (2, 1) & (2, 2) & \cdots & (2, K_a^W) \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ (K_a^H, 0) & (K_a^H, 1) & (K_a^H, 2) & \cdots & (K_a^H, K_a^W) \end{pmatrix} \quad (3)$$

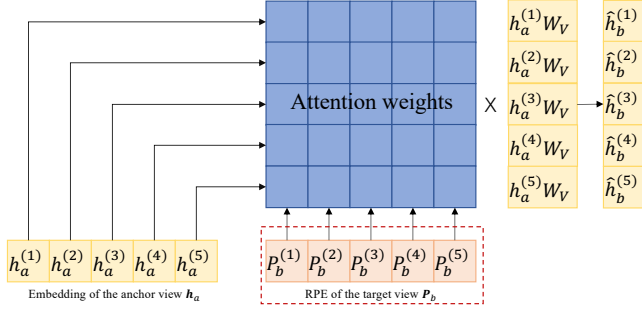


Fig. 3. Illustration of the cross attention mechanism between the relative positional embeddings $\mathbf{P}_b$ and latent anchor embeddings $\mathbf{h}_a$. For better illustrations, we only plot five tokens.

Since we set each grid size of the anchor view as 1. Then the relative positions of the target view becomes:

$$\begin{aligned} \mathbf{rp}_b^{m,n} &= (\mathbf{rp}_b^m, \mathbf{rp}_b^n) \\ &= \underbrace{\left(\frac{K_a \cdot (p_{bi} - p_{ai})}{p_{ah}} + \frac{p_{bh} \cdot K_a \cdot (m-1)}{K_b \cdot p_{ah}} \right)}_{\text{Row}}, \\ &\quad \underbrace{\left(\frac{K_a \cdot (p_{bj} - p_{aj})}{p_{aw}} + \frac{p_{bw} \cdot K_a \cdot (n-1)}{K_b \cdot p_{aw}} \right)}_{\text{Column}} \end{aligned} \quad (4)$$

where $\mathbf{rp}_b^{m,n} \in \mathbb{R}^{1 \times 1 \times 2}$ is the m -th row n -th column vector in the position matrix $\mathbf{rp}_b \in \mathbb{R}^{K_t^H \times K_t^W \times 2}$. Then, for each relative position $\mathbf{rp}_b^{m,n}$, we can compute the relative positional embeddings located in (m, n) by:

$$\mathbf{P}_b^{m,n} = \left[\sin\left(\frac{m}{e^{2*1/d}}\right), \cos\left(\frac{m}{e^{2*2/d}}\right), \dots, \sin\left(\frac{m}{e}\right), \sin\left(\frac{n}{e^{2*1/d}}\right), \cos\left(\frac{n}{e^{2*2/d}}\right), \dots, \sin\left(\frac{n}{e}\right) \right], \quad (5)$$

where $\mathbf{P}_b^{m,n} \in \mathbb{R}^{1 \times 1 \times D}$ is the relative positional embeddings of the patch located at m -th row and n -th column. $e = 10,000$ is the pre-defined hyperparameter, as also commonly used in [19], [23]. $d = \frac{D}{2}$, and D is the hidden dimension of the transformer encoder. By using Eq. 5, we can compute the RPE of each patch, and we denote the relative positional embeddings of all the patches in the target view as $\mathbf{P}_b \in \mathbb{R}^{K_t^H \times K_t^W \times D}$. Finally, we flatten the $\mathbf{P}_b$ to $K_t^H K_t^W \times D$.

B. Forward and Backward Process of Diffusion Models

Given the latent anchor view $\mathbf{z}_a$ and latent target view $\mathbf{z}_b$, and the relative positional embedding $\mathbf{P} \in \mathbb{R}^{K_t^H \times K_t^W \times D}$, we first perform the forward process of diffusion on the target latent view by:

$$\begin{aligned} q(\mathbf{z}_{b_t} | \mathbf{z}_{b_{t-1}}) &= \mathcal{N}(\mathbf{z}_{b_{t-1}} | \sqrt{\alpha_t} \mathbf{z}_{b_{t-1}}, \beta_t \mathbf{I}), \text{ and} \\ q(\mathbf{z}_{b_t} | \mathbf{z}_{b_0}) &= \mathcal{N}(\mathbf{z}_{b_t} | \sqrt{\alpha_t} \mathbf{z}_{b_0}, (1 - \alpha_t) \mathbf{I}), \end{aligned} \quad (6)$$

where $\mathbf{z}_{b_0}$ is the clean latent target view (i.e., $\mathbf{z}_{b_0} = \mathbf{z}_b$) and $\alpha_t = \prod_{i=1}^t \alpha_i$. Then, for the backward process, suppose we have a U-ViT network $\mathbf{U}_\theta$, which is composed of an encoder $\mathbf{E}_\theta$ and a decoder $\mathbf{D}_\theta$. The U-ViT takes the noisy target $\mathbf{z}_{b_t}$, clean latent anchor view $\mathbf{z}_a$ and relative positional embeddings

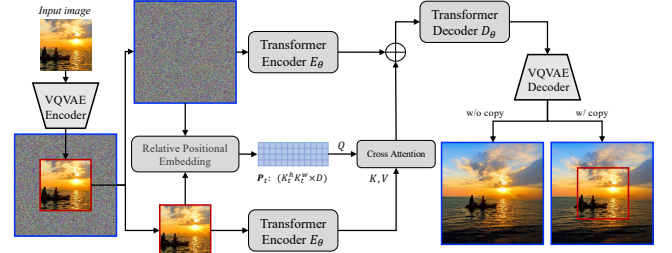


Fig. 4. Sampling pipeline of the proposed EasyOutPainter. Given an input image, we first resize the image to 192x192, followed by the VQVAE encoder E_{VQVAE} . Then, we calculate the relative positional embeddings between the “anchor” (red box) and “target” (blue box) views, and feed them into the trained transformer encoder $\mathbf{E}_\theta$ and decoder $\mathbf{D}_\theta$. Then, we feed the predicted latent embedding to the VQVAE decoder to calculate the generated samples. We illustrate the “copy” and w/o “copy” samples, where they are almost the same, which demonstrates our EasyOutPainter can record the pixel-wise information of the input images.

$\mathbf{P}_b$ as input. Then, the network aims to predict the added noise ϵ_t on the target sequence $\mathbf{z}_{b_t}$. Then, the diffusion objective is:

$$\mathcal{L}_{Diff} = \|\tilde{\epsilon}_t - \epsilon_t\|_p^p, \text{ and } \tilde{\epsilon}_t = f_\theta(\mathbf{z}_a, \mathbf{z}_{b_t}, \mathbf{P}_b, t). \quad (7)$$

where t is the **timestep** in diffusion models. We set $p = 2$ in line with the previous generative methods [37], [38].

C. Position-Aware Transformer and Contrastive Acceleration

Here, we describe the architecture of the U-ViT $\mathbf{U}_\theta$ used in the diffusion model in detail. Consider we have the noisy latent target view $\mathbf{z}_{b_t}$, clean latent target view $\mathbf{z}_{b_0}$, clean latent anchor view $\mathbf{z}_a$, relative positional embeddings $\mathbf{P}_b$ and the timestep t , we first feed the anchor view $\mathbf{z}_a$ and the clean target view $\mathbf{z}_{b_0}$ to the transformer encoder $\mathbf{E}_\theta$ to obtain the latent representation $\mathbf{h}_a \in \mathbb{R}^{K_a^H K_a^W \times D}$ and $\mathbf{h}_b \in \mathbb{R}^{K_b^H K_b^W \times D}$, respectively. Then, the position-aware cross-attention mechanism is proposed to learn positional relationships, which is:

$$\hat{\mathbf{h}}_b = \text{Attn}(\mathbf{Q}_{\mathbf{P}_b}, \mathbf{K}_{\mathbf{h}_a}, \mathbf{V}_{\mathbf{h}_a}) = \text{Softmax}\left(\frac{\mathbf{Q}_{\mathbf{P}_b} \mathbf{K}_{\mathbf{h}_a}^\top}{\sqrt{D}}\right) \mathbf{V}_{\mathbf{h}_a}, \quad (8)$$

where $\hat{\mathbf{h}}_b$ is the predicted latent representation of the target view. $\mathbf{Q}_E = \mathbf{P}_b \mathbf{W}_q$, $\mathbf{K}_{\mathbf{h}_a} = \mathbf{h}_a \mathbf{W}_k$, $\mathbf{V}_{\mathbf{h}_a} = \mathbf{h}_a \mathbf{W}_v$, and $\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v$ are learnable parameters (see Fig. 3). Then, the batch-wise contrastive loss can be formulated as:

$$\begin{aligned} \mathcal{L}_{contra}(\hat{\mathbf{h}}_b^{(i)}) &= -\log \frac{\exp(\hat{\mathbf{h}}_b^{(i)} \cdot \mathbf{h}_b^{(i)} / \tau)}{\sum_j \exp(\hat{\mathbf{h}}_b^{(i)} \cdot \mathbf{h}_b^{(j)} / \tau)} \\ \mathcal{L}_{contra} &= \sum_i \frac{1}{B} \mathcal{L}_{contra}(\hat{\mathbf{h}}_b^{(i)}) \end{aligned} \quad (9)$$

where $\mathbf{h}_b^{(i)}$ and $\mathbf{h}_b^{(j)}$ mean the i -th image and j -th image, respectively. B is the batch size. We also input the noisy latent target view $\mathbf{z}_{b_t}$ to generate the latent embedding $\mathbf{h}_{b_t}$ of the noisy target view. Then, we make a simple element-wise addition on $\mathbf{h}_{b_t}$ and $\hat{\mathbf{h}}_b$, followed by the transformer decoder $\mathbf{D}_\theta$ and a convolutional layer to predict the noise $\tilde{\epsilon}_t$. Finally, the overall loss can be calculated by:

$$\mathcal{L}_{ConOut} = \lambda \cdot \mathcal{L}_{contra} + \mathcal{L}_{Diff} \quad (10)$$

where λ is a hyper-parameter to balance the two losses.

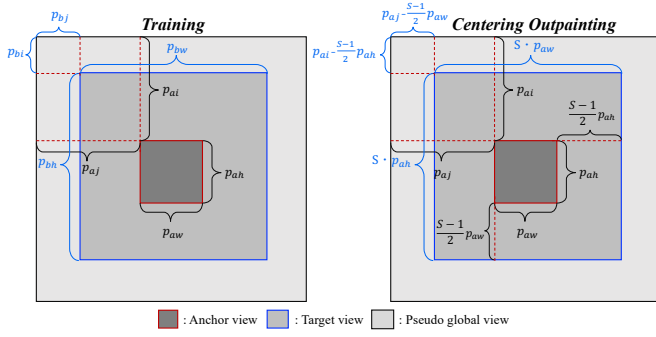


Fig. 5. Illustration of generating the anchor position $\mathbf{p}_a$ and the target position $\mathbf{p}_b$ in the training (left) and **centering** (outpainting the same size around the given sub-image) outpainting (right). For the centering sampling stage, we finally calculate the quadruple $\mathbf{p}_b = (p_{ai} - \frac{S-1}{2} \cdot p_{ah}, p_{aj} - \frac{S-1}{2} \cdot p_{aw}, S \cdot p_{ah}, S \cdot p_{aw})$. The pseudo-global view is designed to simply calculate the relative positions between the anchor and target views.

D. Algorithm for Training and Sampling

We show the training and sampling algorithms in Alg. 1 and Alg. 2, respectively. Note that the conditional ratio p is commonly introduced for classifier-free guidance in previous works [57]. In this work, we simply set all the elements in the $\emptyset$ as 1. For the sampling stage, we first calculate the relative position $\mathbf{rp}_b$ through S (outpainting multiples) by manually setting the $\mathbf{p}_a$ and $\mathbf{p}_b$. For example, for centering outpainting (outpainting the same size around the image, see Fig. 4), we can set the $\mathbf{p}_a$ as $(p_{ai}, p_{aj}, p_{ah}, p_{aw})$. Then, the $\mathbf{p}_b$ can be set as:

$$\mathbf{p}_b = \left(p_{ai} - \frac{S-1}{2} \cdot p_{ah}, p_{aj} - \frac{S-1}{2} \cdot p_{aw}, S \cdot p_{ah}, S \cdot p_{aw} \right) \quad (11)$$

where S is the outpainting multiples. We also illustrate how to calculate $\mathbf{p}_b$ in Fig. 5. Then, we can simply feed the $\mathbf{p}_a$ and $\mathbf{p}_b$ to Eq. 4 to get the relative positions $\mathbf{rp}_b$.

IV. EXPERIMENTS

A. Experiments Setup

Datasets. We use three datasets: Scenery [7], Building Facades [17], and WikiArt [58], in line with [7], [13], [16]. **Scenery** is a natural scenery with diverse natural scenes, consisting of about 5,000 images for training and 1,000 images for testing. **Building Facades** is a city scenes dataset consisting of about 16,000 and 1,500 images for training and testing, respectively. **WikiArt** is a fine-art paintings dataset, which can be obtained from wikiart.org. We use the split manner of genres datasets (used in [13], [17]), which contain 45,503 training images and 19,492 testing images.

Training Details. We implement our approach with PyTorch [59] on a platform equipped with 8 V100 32G GPUs. The encoder is composed of 10 stacked transformer blocks. Then, a cross-attention block is employed, followed by the decoder made of 10 transformer blocks (There are a total of 20 transformer blocks used in EasyOutPainter, where the number of parameters is roughly similar to the previous SOTA QueryOTR [13]). Finally, a 3x3 convolutional layer is adopted to smooth the generated image. In line with

Algorithm 1: Training pipeline of EasyOutPainter.

Input: Raw complete image $\mathbf{x}$, VQVAE encoder: E_{VQVAE} , U-ViT $\mathbf{U}_\theta$ (composed of an encoder: $\mathbf{E}_\theta$ and a decoder $\mathbf{D}_\theta$), max timestep T , conditional ratio p , balance ratio λ .
Result: Trained U-ViT network for outpainting with $\mathbf{E}_\theta$ and $\mathbf{D}_\theta$.

- 1 Initialize parameters θ .
- 2 **repeat**
- 3 Randomly sample timestep t from $1 \sim T$.
- 4 Obtain $\mathbf{x}_a, \mathbf{p}_a$ through RandomCropResize($\mathbf{x}$).
- 5 Obtain $\mathbf{x}_b, \mathbf{p}_b$ through conditioned RandomCropResize($\mathbf{x}$).
- 6 Feed the anchor and target views to the VQVAE encoder to obtain $\mathbf{z}_a$ and $\mathbf{z}_b$, respectively.
- 7 Compute the relative position $\mathbf{rp}_b$ of each patch through Eq. 4.
- 8 Calculate the relative positional embeddings through Eq. 5.
- 9 Feed the latent anchor view and latent target view to the U-ViT encoder $\mathbf{E}_\theta$ to obtain $\mathbf{h}_a, \mathbf{h}_b$.
- 10 Calculate the contrastive loss through Eq. 9.
- 11 Randomly sample Gaussian noise ϵ_t .
- 12 Add noise ϵ_t on the latent target view $\mathbf{z}_b$ to obtain $\mathbf{z}_{b_t}$ via Eq. 6.
- 13 **if** $\text{random.random}() \leq p$ **then**
- 14 $\mathbf{z}_a = \emptyset, \mathbf{P}_b = \emptyset$
- 15 Predict $\tilde{\epsilon}_t$ by $\mathbf{U}_\theta(\mathbf{z}_a, \mathbf{z}_{b_t}, \mathbf{P}_b, t)$.
- 16 Compute the diffusion loss $\|\tilde{\epsilon}_t - \epsilon_t\|_p^p$ via Eq. 7.
- 17 Compute the overall loss by Eq. 10.
- 18 Backward and update the network $\mathbf{U}_\theta$.
- 19 **until** Converge;

Output: g_θ

previous methods [13], we copy the ground truth (input) to the corresponding location in the generated image. We find the copy operation will make a great impact on previous methods, while EasyOutPainter is much more robust to this operation. Therefore, we also report the FID and IS score without the “copy” operations. The number of parameters is approximately equal to QueryOTR [13], which contains 12 transformer blocks in the encoder and 4 transformer blocks in the decoder (they additionally use the decoder in the pix2pixHD [60] and several transformer blocks in the query expansion modules). We adopt AdamW optimizer [61], and we set the learning rate to 0.00015, weight decay to 0.03, and betas to 0.99. In the training stage, we set the random crop ratio of the anchor view to (0.15, 0.6) and the ratio of the target view to (0.6, 1.0), aiming to use the small view to predict the larger view. For all datasets, we train EasyOutPainter 100k iterations with 32 images per GPU on the Scenery, Building Facades, and WikiArt datasets, respectively. Following QueryOTR, we set the resolution of the anchor view as 192x192, and the resolution of the target view is randomly sampled from (192, 208, 224, 240, 256).

Evaluation and Baselines. In line with the literature, we

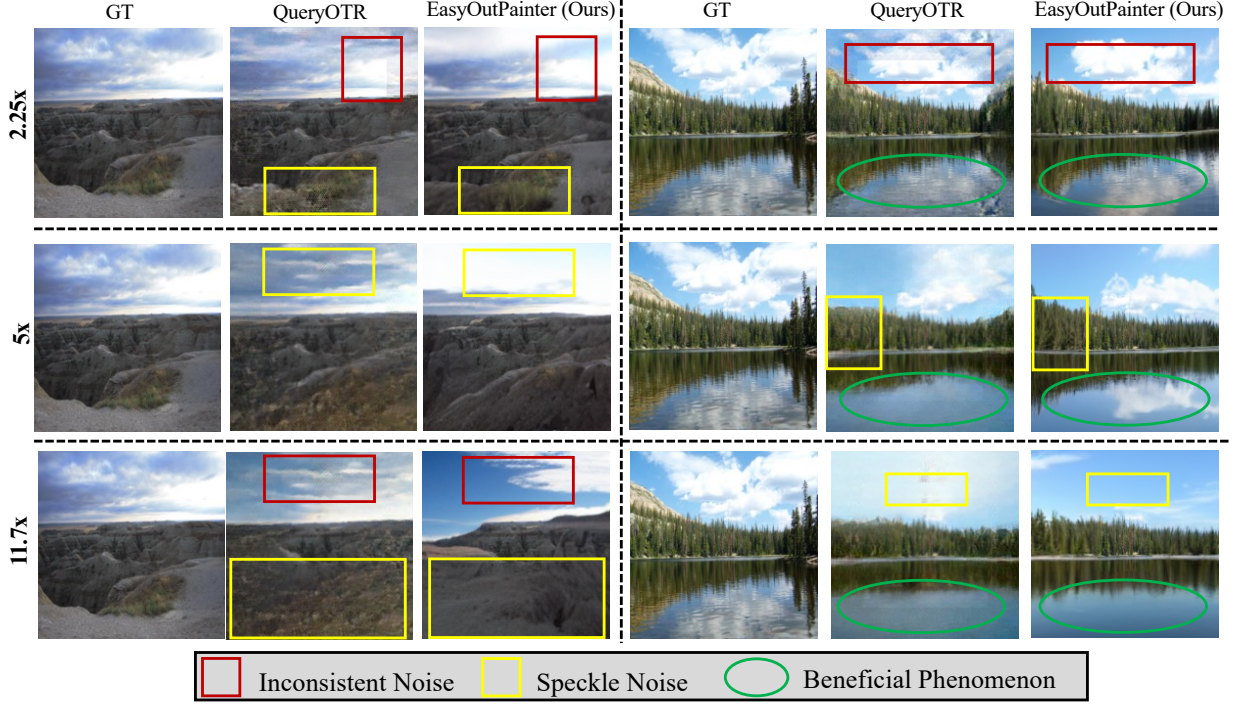


Fig. 6. Comparison on the 2.25x, 5x, and 11.7x settings with the SOTA QueryOTR [13]. The images generated by QueryOTR come from the pre-trained model in their official repository. We highlight two kinds of noises from QueryOTR. The **red** box indicates that the boundary of the input sub-image is inconsistent with the generated region, and a **yellow** box contains noise and spots. We also find some interesting phenomena (highlighted in the **green** ovals of generated images in the right figure), where EasyOutPainter can notice the generated “clouds”, and reflects the “clouds” in “water”. Moreover, the shape of the clouds in the sky and the reflections in the water are also consistent. In contrast, QueryOTR only generates “clouds”, yet ignores the water reflection.

Algorithm 2: Sampling pipeline of EasyOutPainter.

Input: Input sub-image $\mathbf{x}$, the pretrained network $\mathbf{U}_\theta$, VQVAE encoder and decoder E_{VQVAE}, D_{VQVAE} , max timestep T , the specified outpainting multiple S , the specified outpainting resolutions P , classifier-free guidance ratio w .

Result: Outpainted image $\mathbf{y}$.

- 1 Compute the relative position $\mathbf{rp}_b$ of $\tilde{\mathbf{y}}$ through S and P by Eq. 11 (Also illustrated in Fig. 5).
- 2 Acquire the positional embedding $\mathbf{P}_b$ through $\mathbf{p}_b$.
- 3 Randomly sample Gaussian noise $\mathbf{y}_T$.
- 4 **repeat**
- 5 Acquire latent embeddings $\mathbf{z}_a \leftarrow E_{VQVAE}(\mathbf{x})$.
- 6 Predict $\tilde{\epsilon}_T$ by $\mathbf{U}_\theta(\mathbf{z}_a, \mathbf{y}_T, \mathbf{P}_b, T)$.
- 7 Predict unconditional noise $\tilde{\epsilon}_T^u$ by $\mathbf{U}_\theta(\emptyset, \mathbf{y}_T, \emptyset, T)$.
- 8 Perform classifier-free guidance $\tilde{\epsilon}_T = (w + 1) \cdot \tilde{\epsilon}_T - w \cdot \tilde{\epsilon}_T^u$.
- 9 Compute $\tilde{\mathbf{y}}_0$ and $\mathbf{y}_{T-1}$.
- 10 $T \leftarrow T - 1$.
- 11 **until** $T \geq 1$;
- 12 $\mathbf{y} \leftarrow D_{VQVAE}(\mathbf{y}_0)$.

Output: $\mathbf{y}$.

(FID) [63] to measure the generative quality. Note that the upper bounds of IS are 4.092, 5.539, and 8.786 for Scenery, Building Facades, and WikiArt, respectively, which are calculated by real images in the test set. Here we do **not** use PSNR (peak signal-to-noise ratio) as once used in outpainting because PSNR cannot reflect the **diversity** of generated images, which is important for generative models, and more details are given in Sec. IV-C. Alternatively, we report the PSNR score between the input sub-images and the center region of the generated images. Because we think this score is more meaningful to detect whether the network ignores the input conditions. We make comparisons with five SOTA image outpainting methods, NSIPO [7], SRN [32], IOH [16], Uformer [17] and QueryOTR [13].

Sampling Details. In line with the previous method [13], [17], for the testing stage, all images are resized to 192x192 as the ground truth, and then the input sub-images are obtained by center cropping to the sizes 128x128, 86x86, and 56x56 for 2.25x, 5x, and 11.7x outpainting, respectively. Then, all the sub-images are resized to 192x192 and fed to as the anchor view. The total output sizes are 2.25, 5, and 11.7 times the input namely 2.25x, 5x, and 11.7x outpainting, respectively. We set the classifier-free guidance ratio as 1.3 by default.

B. Experimental Results

Quantitative Results. Table II shows that the proposed EasyOutPainter with a copy operation can outperform previous SOTA in most settings. In particular, with a larger outpainting

use Inception Score (IS) [62], Frechet Inception Distance

TABLE II

RESULTS AT DIFFERENT MULTIPLES. 2.25X, 5X, AND 11.7X MEAN INPUT SIZES ARE 128x128, 86x86, AND 56x56, RESPECTIVELY. AFTER FEEDING TO THE NETWORK g_θ , ALL INPUT SUB-IMAGES ARE FIRST RESIZED TO 192x192. “+ COPY” OPERATION MEANS AFTER THE NETWORK YIELDS THE EXTRAPOLATED IMAGES, THE MODEL COPIES THE INPUT SUB-IMAGES TO THE CORRESPONDING CENTER REGION IN THE EXTRAPOLATED IMAGES. THE BEST RESULTS AND SECOND BEST RESULTS ARE INDICATED BY **BOLDFACE** AND UNDERLINE, RESPECTIVELY.

Multiple	Methods	Type	Encoder	Scenery		Building Facades		WikiArt	
				FID ↓	IS ↑	FID ↓	IS ↑	FID ↓	IS ↑
2.25x	SRN [32]	GAN-based	VGG [64]	47.781	2.981	38.644	3.862	76.749	3.629
	NSIPO [7]	GAN-based	ResNet [15]	25.977	3.059	30.465	4.153	22.242	5.600
	IOH [16]	GAN-based	Conv layers	32.107	2.886	49.481	3.924	40.184	4.835
	Uformer [17]	GAN-based	Swin Transformer [18]	20.575	3.249	30.542	4.189	15.904	6.567
	QueryOTR [13] + Copy	GAN+MAE	ViT [20]	<u>20.366</u>	3.955	<u>22.378</u>	<u>4.978</u>	<u>14.955</u>	<u>7.896</u>
	EasyOutPainter (Ours)	Diffusion-based	ViT [20]	26.460	3.674	29.058	4.694	15.552	7.427
	EasyOutPainter + Copy (Ours)	Diffusion-based	ViT [20]	18.255	<u>3.888</u>	19.065	5.366	11.516	8.768
5x	SRN [32]	GAN-based	VGG [64]	83.772	2.349	74.304	3.651	137.997	3.039
	NSIPO [7]	GAN-based	ResNet [15]	45.989	2.606	58.341	3.669	51.668	4.591
	IOH [16]	GAN-based	Conv layers	44.742	2.655	76.476	3.456	75.070	4.289
	Uformer [17]	GAN-based	Swin Transformer [18]	39.801	2.920	63.915	3.798	41.107	5.900
	QueryOTR [13] + Copy	GAN+MAE	ViT [20]	39.237	3.431	41.273	4.547	43.757	6.341
	EasyOutPainter (Ours)	Diffusion-based	ViT [20]	<u>31.565</u>	<u>3.701</u>	<u>34.181</u>	4.467	<u>22.508</u>	<u>6.951</u>
	EasyOutPainter + Copy (Ours)	Diffusion-based	ViT [20]	26.885	3.743	29.186	4.818	20.597	7.912
11.7x	SRN [32]	GAN-based	VGG [64]	115.193	2.087	110.036	2.938	181.533	2.504
	NSIPO [7]	GAN-based	ResNet [15]	64.457	2.405	81.301	3.431	75.785	4.225
	IOH [16]	GAN-based	Conv layers	58.629	2.432	95.068	2.790	108.328	3.728
	Uformer [17]	GAN-based	Swin Transformer [18]	60.497	2.638	93.888	3.388	72.923	5.904
	QueryOTR [13] + Copy	GAN+MAE	ViT [20]	60.977	3.114	64.926	4.612	69.951	5.683
	EasyOutPainter (Ours)	Diffusion-based	ViT [20]	<u>37.503</u>	<u>3.465</u>	<u>38.926</u>	4.433	<u>28.418</u>	<u>6.623</u>
	EasyOutPainter + Copy (Ours)	Diffusion-based	ViT [20]	33.866	3.544	35.962	<u>4.577</u>	27.677	7.251

multiple, it can obtain better FID and IS scores. For 11.7x outpainting, it improves the previous SOTA QueryOTR [13] **27.111**, **28.964**, and **42.274** FID scores on Scenery, Building and WikiArt datasets, respectively. We also find on the Wikiart dataset, our EasyOutPainter achieves 8.768 FID scores under 2.25x settings, which is very close to the ground truth (real image, 8.786 FID), and demonstrates it can generate realistic images conditioned by the given sub-images.

Qualitative Results. Examples of visual results are shown in Fig. 6 (the “copy” operation is added on all methods for better comparisons). EasyOutPainter effectively outpaints the images by querying the global semantic-similar image patches. As seen from the 2.25x outpainting results, EasyOutPainter could generate more realistic images with vivid details and enrich the contents of the generated regions. In addition, although QueryOTR [13] adds the smoothing module to handle the inconsistency of the boundary region, there are still a few noisy spots as highlighted in **red** boxes. For 5x and 11.7x results, we can clearly see the images generated from the QueryOTR are much vaguer than those generated by EasyOutPainter. Moreover, EasyOutPainter can handle details well. For example, the “clouds’ reflection” in the “water” is consistent with the generated “clouds” in the sky, as highlighted in the **green** ovals. We also provide more qualitative comparisons and generated images.

Sampling (i.e. Generation) Speed. We also compare the sampling speed of EasyOutPainter with QueryOTR [13]. Specifically, we first train EasyOutPainter with 100,000 iterations. Then, for the sampling stage, we evaluate the pre-trained EasyOutPainter with different timesteps. Table III reports the wall-clock time spent on generating 64 images with 20 timesteps on a single V100 GPU. We report the results of both

TABLE III

COMPARISON OF SAMPLING TIME, FID, AND INCEPTION SCORES WITH 2.25X, 5X, 11.7X SETTINGS ON SCENERY DATASET. THE SAMPLING TIME IS THE WALL-CLOCK TIME OF GENERATING 64 IMAGES ON A SINGLE GPU. “CFG” MEANS CLASSIFIER-FREE GUIDANCE (THE GUIDANCE RATIO IS SET AS 1.3 AND THE TIMESTEPS FOR SAMPLING IS SET AS 20).

Method	Multiples	Time (Sec.) ↓	FID ↓	IS ↑
QueryOTR [13]	2.25x	23.728	20.366	3.955
EasyOutPainter (ours w/ cfg)		20.185	18.301	3.896
EasyOutPainter (ours w/o cfg)		10.929	18.845	3.744
QueryOTR [13]	5x	47.456	39.237	3.431
EasyOutPainter (ours w/ cfg)		<u>20.185</u>	<u>26.379</u>	3.696
EasyOutPainter (ours w/o cfg)		10.929	26.068	<u>3.557</u>
QueryOTR [13]	11.7x	71.184	60.977	3.114
EasyOutPainter (ours w/ cfg)		<u>20.185</u>	32.341	3.486
EasyOutPainter (ours w/o cfg)		10.929	<u>32.997</u>	<u>3.367</u>

w/ and w/o classifier-free guidance, where both two strategies cost less wall-clock time than previous SOTA QueryOTR [13], especially for 5x and 11.7x settings (Since EasyOutPainter can outpaint images with any multiples in one step, the cases for 2.25x, 5x, and 11.7x spend almost the same time. In contrast, QueryOTR will take much more time under the 5x and 11.7x settings than in 2.25x). Specifically, under 2.25x, 5x, and 11.7x settings, our EasyOutPainter (w/o cfg) only spends **46.059%**, **23.029%**, and **15.353%** wall-clock time (seconds) of the previous SOTA QueryOTR [13].

C. Ablation Studies

Outpainting in Arbitrary Positions. Previous outpainting methods mainly plot the same multiples around the top, down, left, and right regions, and they can simply find where the input sub-image should be located in the generated image.

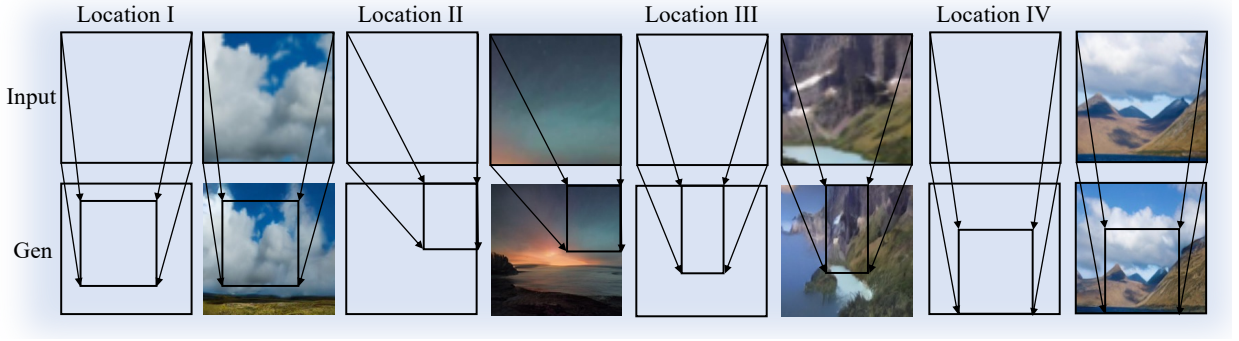


Fig. 7. Example images generated by EasyOutPainter (with same resolutions) via random relative positional embedding. The input image is mapped to the corresponding location in the generated image (Gen). Note that the generated images do not explicitly undergo any “copy” operation, but still reserve the input pixels. Moreover, EasyOutPainter learns the scales of input images according to the given mode setting.

TABLE IV

COMPARISONS OF THE CENTER PSNR SCORE OVER 2.25x, 5x, AND 11.7x. NOTE THAT WE FIND EASYOUTPAINTER CALCULATES SEVERAL INFINITE VALUES, WHICH INDICATES THE CENTER PARTS IN IMAGES GENERATED BY EASYOUTPAINTER ARE COMPLETELY THE SAME WITH THE INPUT SUB-IMAGES WITHOUT ANY BIAS. HENCE, TO MAKE OUR MODEL COMPARABLE, WE ONLY INCLUDE THE IMAGES WHOSE PSNR SCORES ARE BELOW 1,000. FOR QUERYOTR, WE DO NOT FIND THE INFINITE VALUE PHENOMENON, AND WE DIRECTLY REPORT THE AVERAGED PSNR SCORES.

Method	Dataset	2.25x	Settings 5x	11.7x
QueryOTR [13]	Scenery	22.146	18.926	15.375
EasyOutPainter (Ours)		28.759 (+6.613)	27.990 (+9.064)	27.344 (+11.969)
QueryOTR [13]	Buildings	18.591	15.318	13.119
EasyOutPainter (Ours)		25.175 (+6.584)	24.713 (+9.395)	23.749 (+10.630)
QueryOTR [13]	Wikiart	19.726	15.874	14.016
EasyOutPainter (Ours)		26.027 (+6.301)	25.606 (+9.732)	25.071 (+11.055)

Thus, the “copy” operation can be simply finished. However, for outpainting in random positions, it is difficult to find the corresponding locations, since the two images (generated and original) could have different scales. Moreover, the two images may also not intersect. Hence, we directly illustrate the images generated by EasyOutPainter without the “copy” operation. Some examples generated from the controlled position are shown in Fig. 7, where the images generated by EasyOutPainter without the “copy” iteration can be still vivid and realistic. Furthermore, without the “copy” operation, the generated images can also record the pixel information and put it into the corresponding locations. Meanwhile, as the scales of input sub-images and generated images can vary, it implicitly learns to scale the input sub-images as well.

Diversity of Generated Images and PSNR in Center Regions. We also show five generated images with fixed positions and input in Fig. 8, showing EasyOutPainter can generate diverse content in the generated regions. Furthermore, it also retains the input pixels in the center parts of generated samples. Recall that in Sec. IV-A, we choose not to use the PSNR score as evaluation metrics, as we also need to account for the diversity. Here we provide further analysis with the PSNR score whose definition is as follows, and a higher score

suggests a smaller mean square error:

$$PSNR(\mathbf{x}, \mathbf{y}) = 10 \cdot \log_{10} \left(\frac{MAX_{\mathbf{x}}^2}{\mathbb{E}_{i,j} [\mathbf{x}_{i,j} - \mathbf{y}_{i,j}]^2} \right), \quad (12)$$

s.t. $0 \leq i, j \leq H, W$

where $\mathbf{x}, \mathbf{y}$ are input images, and H, W are the height and width of the input images, and $MAX_{\mathbf{x}}^2$ is a constant. We then report the PSNR score of the generated center region and input images in Table IV. Note that since the objective of QueryOTR is only added to the generated position, the center part is not regularized in the training stage. Hence, the generated center content is completely black area. Hence, for better comparison, we modify QueryOTR, adding the objective to the whole generated images with the $\mathcal{L}_{rec}$ loss. The PSNR scores in Table IV and the generated samples in Fig. 8 demonstrate our model can: 1) memorize the input pixels and put them into corresponding locations; 2) learn the semantic information of input images, and outpaint the image with consistent contents around the images. It is worth noting that the central regions in images generated by our EasyOutPainter without the “copy” operation can be completely the same with input images ($\mathbb{E}_{i,j} [\mathbf{x}_{i,j} - \mathbf{y}_{i,j}]^2 = 0$), which indicates, our EasyOutPainter can reconstruct the images without any bias in some cases. Since these cases without bias would lead to infinite values of the PSNR score, which will impact the average values, we directly remove these infinite values. Experimental results show even if we remove these infinite values, our method can still produce much higher center PSNR scores than QueryOTR.

Impact of Contrastive Loss. Since we pre-train the U-ViT U_{θ} from scratch, we design the contrastive loss to speed up the convergence of the U-ViT transformer encoder E_{θ} . However, we find that when the contrastive loss is added in the whole training iterations, the contrastive objective would heavily influence the diffusion trajectory, and generate images in different distributions (both the FID and IS scores in the Table V are extremely high). Therefore, we only add the contrastive loss in some previous iterations. Table V shows the results with the flexible and fixed contrastive loss, where “fixed” means we add the contrastive loss in the whole training iterations with $\lambda = 0.5$, while “flexible” means we only add the contrastive loss in some previous iterations (Specifically,



Fig. 8. Examples outpainted by EasyOutPainter with the same inputs of 5 trials, where EasyOutPainter always generated appropriate but diverse contents. Top and bottom figures are sampled in the Scenery, and Wikiart datasets, respectively.

TABLE V

FID AND IS SCORES ON THE SCENERY DATASET WITH FLEXIBLE AND FIXED CONTRASTIVE LOSS IN EQ. 9. “FLEXIBLE” MEANS CHANGING λ OVER ITERATIONS, AND “FIXED” MEANS FIXING λ IN TRAINING.

Method	Multiples	Metric	
		FID ↓	IS ↑
w/o $\mathcal{L}_{contra}$ + “copy”	2.25x	19.871	3.819
w/ fixed $\mathcal{L}_{contra}$ + “copy”		39.793 (- 19.922)	3.958 (+ 0.139)
w/ flexible $\mathcal{L}_{contra}$ + “copy”		18.255 (+ 1.616)	3.888 (+ 0.069)
w/o $\mathcal{L}_{contra}$ + “copy”	5x	28.374	3.669
w/ fixed $\mathcal{L}_{contra}$ + “copy”		54.324 (- 25.950)	3.860 (+ 0.191)
w/ flexible $\mathcal{L}_{contra}$ + “copy”		26.885 (+ 1.489)	3.743 (+ 0.074)
w/o $\mathcal{L}_{contra}$ + “copy”	11.7x	36.177	3.498
w/ fixed $\mathcal{L}_{contra}$ + “copy”		82.053 (- 45.876)	3.979 (+ 0.481)
w/ flexible $\mathcal{L}_{contra}$ + “copy”		33.866 (+ 2.311)	3.544 (+ 0.046)

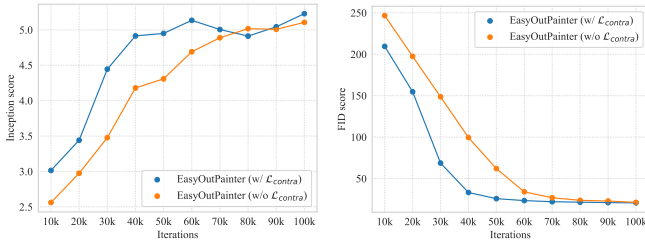


Fig. 9. Comparisons of inception (left) and FID (right) scores on the Facades dataset with and without contrastive loss in Eq. 9 over training iterations.

we added a cosine schedule to the contrastive loss weight. This means that for the first half of the training iterations, we set the balancing coefficient $\lambda = \lambda_{base} \cdot \cos(\frac{Iter}{Max_Iter}\pi)$, where λ_{base} is set to 0.5. Here, Max_Iter represents the maximum number of training iterations, and $Iter$ denotes the current training iterations. When $Iter \geq Max_Iter/2$, we set $\lambda = 0$. We find that with flexible contrastive loss, our EasyOutPainter can benefit from both FID and IS. Fig. 9 shows the FID and IS curve with training iterations, where EasyOutPainter (w/ $\mathcal{L}_{contra}$) consistently converges faster than EasyOutPainter (w/o $\mathcal{L}_{contra}$).

Impact of Multi-Scale Training Strategy. One of the main contributions of EasyOutPainter is that we introduce the

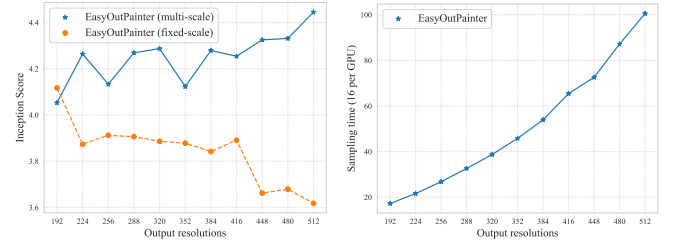


Fig. 10. Inception score and sampling times (sec.) with different output resolutions on the Scenery dataset. Note that when the output resolution is larger than 384, it would bring the OOM (Out of Memory) error. Therefore, for settings that output resolution is larger than 384, we set the batch size as 8, and report the double time of each batch.

multi-scale training strategy, which enables EasyOutPainter to outpaint images with continuous resolutions. To better demonstrate the effectiveness of the multi-scale training strategy, we outpaint images with different output resolutions. Specifically, we keep the resolutions of the input sub-image as 192x192, and outpaint the images with (192x192~512x512) resolutions. The generated regions have the same ratio of the input sub-image. For example, when output images with 192x192, EasyOutPainter do not output image (since the input sub-image also has the 192x192 resolutions), but perform the role of VAE; and when output images with 384x384, the center (192x192) regions should be the same with the sub-images. Fig. 10 illustrates the inception scores under multi-scale training and fixed-scale training strategies. We find that multi-scale training outperforms fixed-scale training with a large range except for output resolution equal to 192, which indicates EasyOutPainter (multi-scale) is much more robust for output resolutions than EasyOutPainter (fixed-scale). We also surprisingly find that when outputting with larger resolutions, our EasyOutPainter achieves higher inception scores (4.269 for output resolution 288x288 (288/192 = 1.5)), which greatly outperforms previous methods [13] (3.955 for output resolution 192x192 (192/128=1.5)).

Impact of Random Crop Ratio. For better consistency of inputs in the training stage and sampling stage, we crop

TABLE VI

INCEPTION SCORES ON BUILDING FACADES DATASET WITH DIFFERENT OUTPAINTING MULTIPLES AND CROP RATIOS. FOR EACH MODEL OF EASYOUTPAINTER, WE TRAIN 100,000 ITERATIONS, AND SAMPLE IMAGES WITH CFG RATIO $w = 1.3$.

Method	Crop ratio	Outpainting multiples		
		2.25x	5x	11.7x
QueryOTR [13]	~	4.978	4.547	4.612
	(0.05, 0.05)	4.628 (-0.350)	4.583 (+0.026)	4.618 (+0.006)
EasyOutPainter	(0.05, 0.15)	4.971 (-0.007)	4.667 (+0.120)	4.701 (+0.089)
	(0.05, 0.25)	5.194 (+0.216)	4.767 (+0.220)	4.668 (+0.056)
	(0.05, 0.35)	5.261 (+0.283)	4.782 (+0.235)	4.596 (-0.016)
	(0.05, 0.40)	5.366 (+0.388)	4.818 (+0.271)	4.577 (-0.035)

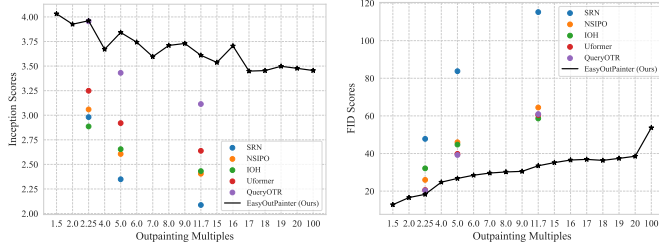


Fig. 11. Inception and FID scores on the Scenery dataset with different outpainting multiples, where our EasyOutPainter achieves state-of-the-art results under almost all settings. Note that since EasyOutPainter can additionally outpaint images with continuous multiples, we show the results of EasyOutPainter as a continuous curve instead of scatter plots.

the view x_b with a larger crop ratio than view x_a (since the outpainted images are usually larger than input images). We conduct experiments to analyze the effect of random crop ratios. Specifically, we fix the crop ratio of target view x_b as (0.4, 1.0), and switch the crop ratio of anchor views x_a from 0.05~0.40. We train the model with 100,000 iterations on the Scenery dataset and show the results in Table VI. The results show the query crop ratio influences 2.25x, 5x, and 11.7x experiments in different manners. Specifically, for 2.25x experiments, since the input images are 128x128, and the outpainted images are 192x192, where the extrapolated image is only a little larger than the input sub-image. Hence, EasyOutPainter with larger crop ratios outperforms that with smaller ones. For 5x and 11.7x experiments, the size of extrapolated images is much larger than the given input sub-image. As a result, EasyOutPainter with smaller crop ratios outperforms that with larger ones. On top of that, we also find when the anchor crop ratio equals 0.05, the inception score drops with a large range, and we guess that is because when the random crop ratio is set (0.05, 0.05), and we always feed the images with the same scales to EasyOutPainter, and correspondingly, it can not learn to scale images. Then, in the sampling stage, since images in the test set are usually in different scales, it is difficult for EasyOutPainter to handle the scaling gaps between the training stage and the testing stage.

Impact of Variant Outpainting Multiples. Since our EasyOutPainter can outpaint images with arbitrary and continuous multiples, we test the pretrained EasyOutPainter with variant outpainting multiples. Following the settings in the main results, we fix the CFG ratio as 1.3, and use 50 timesteps to generate images. Fig. 11 illustrates the FID and

TABLE VII

PERFORMANCE OF PLAIN DIFFUSION MODELS AND PQGAN (INTEGRATE PQ SCHEME IN GAN) ON THE SCENERY DATASET WITHOUT THE “COPY” OPERATION (AS WE REPORT THE CENTER PSNR SCORE).

Method	Multiples	Metric		
		FID ↓	IS ↑	Center PSNR ↑
Valinna diffusion [21]	2.25x	57.425	3.816	11.538
PQGAN (Ours)		32.139	3.811	27.097
EasyOutPainter (Ours)		26.460	3.674	28.759
Valinna diffusion [21]	5x	59.475	3.286	10.894
PQGAN (Ours)		38.466	3.216	26.893
EasyOutPainter (Ours)		31.565	3.701	27.990
Valinna diffusion [21]	11.7x	73.587	2.867	10.347
PQGAN (Ours)		49.943	2.853	24.139
EasyOutPainter (Ours)		37.503	3.465	27.344

TABLE VIII

IMPACT OF DIFFERENT TYPES OF POSITIONAL EMBEDDINGS ON SCENERY. ALL METHODS ARE WITHOUT “COPY” OPERATION. THE SIN-COS POSITIONAL EMBEDDING IS ALSO USED IN THE MAIN COMPARISONS (TABLE II).

RPE Type	Multiples	Metric		
		FID ↓	IS ↑	Center PSNR ↑
None	2.25x	60.378	3.823	10.564
Learnable		32.542	3.806	27.690
Sin-Cos (Ours)		18.255	3.888	28.759
None	5x	77.138	3.291	10.299
Learnable		36.812	3.391	27.169
Sin-Cos (Ours)		26.885	3.743	27.990
None	11.7x	91.154	3.014	10.297
Learnable		48.392	3.207	24.559
Sin-Cos (Ours)		33.866	3.544	27.344

IS comparisons of variant baselines on the Scenery dataset. Where our EasyOutPainter achieves state-of-the-art FID on all settings. Besides, EasyOutPainter also obtains 3.961 inception score on 2.25x settings, which is only slightly lower than QueryOTR [13] (For all other settings, our EasyOutPainter consistently outperforms the previous methods). Besides, we further test our EasyOutPainter with 100x outpainting settings, where our method obtains 3.455 IS and 53.688 FID scores, which is also better than all previous methods under 11.x settings (60.977 IS and 3.114 FID score).

Integrate PQ Scheme Into Other Generative Models. Beyond diffusion, we also integrate our PQ learning paradigm into GAN-based model. In addition, to analyze the effect of the positional query, we also report the performance when directly using diffusion models [21]. Table VII shows the results. Specifically, we find the quality of the images generated by GAN is lower than the diffusion model [21], as the inception score of PQGAN is lower than diffusion models [21]. However, the diffusion model is not well conditioned by the input sub-image (the FID and Center PSNR scores are much worse than PQGAN and EasyOutPainter). The high FID and Center PSNR scores indicate the proposed PQ scheme can provide a strong condition, enhancing the generative models [14], [21] to learn where to outpaint.

Impact of the Positional Embedding. To analyze the

TABLE IX

PERFORMANCE WHEN TRANSFERRING THE MODEL TO DIFFERENT EVALUATION DISTRIBUTIONS. “INNER DIST” MEANS EVALUATING THE MODEL UNDER THE SAME SETTINGS AS THE TRAINING STAGE. “OUTER DIST” MEANS EVALUATING THE MODEL UNDER DIFFERENT SETTINGS FOR THE TRAINING STAGE. WE USE THE SAME BACKBONE AND TRAINING RECIPE OF PQDIFF [22] (I.E. OUR CONFERENCE VERSION) IN THIS TABLE.

Metric	Method	Inner dist		
		2.25x	5x	11.7x
IS ($\uparrow$)	Discrete	3.813	3.553	3.207
	EasyOutPainter (Ours)	3.849	3.547	3.269
FID ($\downarrow$)	Discrete	29.481	33.917	47.239
	EasyOutPainter (Ours)	29.446	34.492	44.517
Center PSNR ($\uparrow$)	Discrete	27.814	27.017	23.917
	EasyOutPainter (Ours)	27.676	27.267	24.697
Metric	Method	Outer dist		
		4x	9x	16x
IS ($\uparrow$)	Discrete	3.626	3.391	2.891
	EasyOutPainter (Ours)	3.827	3.435	3.012
FID ($\downarrow$)	Discrete	35.183	50.183	53.174
	EasyOutPainter (Ours)	31.274	39.927	49.271
Center PSNR ($\uparrow$)	Discrete	18.271	17.238	14.927
	EasyOutPainter (Ours)	27.477	25.820	23.175

effect of the positional embedding, we conduct a group of experiments with different types of positional embeddings. We mainly consider two types of embeddings (sin-cos and learnable). Learnable embedding means we take the relative position (m, n) as input and use an MLP composed of two linear layers with the activation function to map the 2-dimensions to D -dimensions. Table VIII shows the results with different positional embedding. Note that None means without relative positional embedding. Thus, the model can not learn the positional relationships between input sub-image and extrapolated images. Correspondingly, the inception score only drops with a little range, but the FID and Center PSNR drop with a large range.

Impact of the Continuous and Discrete Positional Embeddings. To further explore how EasyOutPainter learns the relative position, we conduct a set of ablation studies. Specifically, we remove the randomness of the positions of the anchor views and target views. Specifically, the anchor view is the center region of the target view, and the target view is set 2.25x, 5x, and 11.7x larger than the anchor view with the same probability at each training iteration (1/3 for each multiple) to better compare with the main results in our paper. Therefore, the sizes of the anchor view and target view are discrete, as they are fixed as three numbers. We train the Discrete version on 8 V100 GPUs with 80k iterations (in line with the main results). Table IX shows the results when evaluating with variant outpainting multiples. We find under 2.25x, 5x, and 11.7x settings (inner distribution), discrete training, and achieve similar results with EasyOutPainter. However, when transferred to other outpainting multiples (4x, 9x, 16x), both three scores of discrete training drop, especially for Center PSNR. We guess the reason behind the interesting phenomenon is that the “Discrete training” strategy only memorizes the pixel information of the input sub-image,

TABLE X

COMPARISON OF DIFFERENT TRAINING OBJECTIVES ON THE SCENERY DATASET. SINCE THE COPY OPERATION WOULD LEAD TO THE INFINITE VALUE IN THE CENTER PSNR SCORE, WE ONLY REPORT THE CENTER PSNR SCORE WITHOUT THE COPY OPERATION. 2.25x, 5x, AND 11.7x FOLLOW THE SETTING IN THE MAIN EXPERIMENTS. “FM” MEANS USING THE FLOW MATCHING OBJECTIVE [65].

Method	Multiples	Metrics		
		FID $\downarrow$	IS $\uparrow$	Center PSNR $\uparrow$
QueryOTR [13] w/ copy	2.25x	20.366	3.955	$\sim$
x_0 pred w/o copy		61.097	3.451	$\sim$
x_0 pred w/ copy		32.867	3.783	23.937
FM w/o copy		25.185	3.628	27.773
FM w/ copy		18.576	3.869	$\sim$
Noise pred w/o copy		26.460	3.674	28.759
Noise pred w/ copy		18.255	3.888	$\sim$
QueryOTR [13] w/ copy	5x	39.237	3.431	$\sim$
x_0 pred w/o copy		51.314	3.677	$\sim$
x_0 pred w/ copy		67.980	3.308	23.158
FM w/o copy		33.152	3.739	28.013
FM w/ copy		27.198	3.772	$\sim$
Noise pred w/o copy		31.565	3.701	27.990
Noise pred w/ copy		26.885	3.743	$\sim$
QueryOTR [13] w/ copy	11.7x	60.977	3.114	$\sim$
x_0 pred w/o copy		47.117	3.116	$\sim$
x_0 pred w/ copy		53.148	2.972	25.198
FM w/o copy		38.184	3.441	26.989
FM w/ copy		32.917	3.499	$\sim$
Noise pred w/o copy		37.503	3.465	27.344
Noise pred w/ copy		33.866	3.544	$\sim$

TABLE XI

COMPARISON OF SAMPLING TIME, FID, AND INCEPTION SCORES WITH 2.25x, 5x, 11.7x SETTINGS ON SCENERY DATASET. SAMPLING TIME IS THE WALL-CLOCK TIME OF GENERATING 64 IMAGES ON A SINGLE GPU. “CFG” MEANS CLASSIFIER-FREE GUIDANCE (WITH RATIO 1.3).

Multiples	Method	Timesteps	Metrics		
			Time (Sec.) $\downarrow$	FID $\downarrow$	IS $\uparrow$
2.25x	QueryOTR [13]	$\sim$	23.728	20.366	3.955
	EasyOutPainter (w/ CFG)	15	16.559	18.869	3.878
	EasyOutPainter (w/o CFG)	15	9.059	19.541	3.779
	EasyOutPainter (w/ CFG)	20	20.185	18.301	3.896
	EasyOutPainter (w/o CFG)	20	10.929	18.845	3.744
5x	QueryOTR [13]	$\sim$	47.456	39.237	3.431
	EasyOutPainter (w/ CFG)	15	16.559	26.988	3.647
	EasyOutPainter (w/o CFG)	15	9.059	26.772	3.559
	EasyOutPainter (w/ CFG)	20	20.185	26.379	3.696
	EasyOutPainter (w/o CFG)	20	10.929	26.068	3.557
11.7x	QueryOTR [13]	$\sim$	71.184	60.977	3.114
	EasyOutPainter (w/ CFG)	15	16.559	33.744	3.427
	EasyOutPainter (w/o CFG)	15	9.059	32.529	3.363
	EasyOutPainter (w/ CFG)	20	20.185	32.341	3.486
	EasyOutPainter (w/o CFG)	20	10.929	32.997	3.367

but fails to scale the sub-image with proper ratio due to the outpainting multiples gaps in the training and inference stage.

Impact of different objectives. To better explore the impact of different diffusion objectives on our method, we applied our approach to various techniques. Specifically, for diffusion, we experimented with methods that predict x_0 and methods that predict noise. Additionally, we tested the framework using rectified flow [65]. Table X presents the evaluation metrics for FID, IS, and Center PSNR. We observed that, consistent with previous findings [21], [38], predicting noise yielded significantly better results than predicting x_0 . Furthermore, Flow Matching (20 timesteps sampling) achieved comparable results to predicting noise.

Sampling Timesteps. For better comparisons, we provide

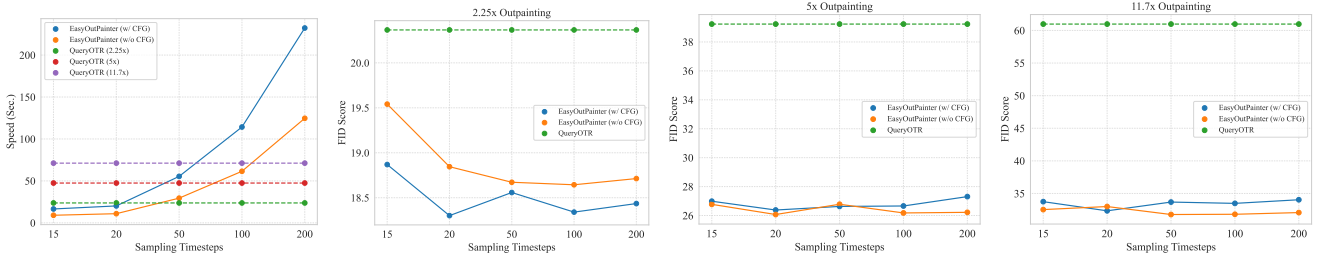


Fig. 12. Comparisons of sampling times (left plot) and FID score (right three plots) with different timesteps on the Scener dataset. “CFG” means classifier-free guidance. For all experiments, we train EasyOutPainter on Scener with 100k iterations. Then we generate images under the settings (2.25x, 5x, 11.7x).

results of EasyOutPainter with variant timesteps in Fig. 12 and Table XI. Specifically, in the training phase, we set timesteps as 1,000. In the testing phase, we change the timesteps from 15~200 (since we observe that with larger timesteps, the IS and FID scores won’t improve anymore). We find the “CFG” would bring more computational cost than without “CFG”, which is because “CFG” would compute the null-condition, and feed them into the U-ViT U_θ . Besides, we find when we set sampling timesteps less than 20, our EasyOutPainter (both w/ and w/o classifier-free guidance.) could spend less time than QueryOTR [13]. Specifically, under 2.25x, 5x, and 11.7x settings, our EasyOutPainter (15 timesteps w/o “CFG”) achieves higher scores, and only spends about 33%, 19%, and 13% wall-times (seconds) of QueryOTR [13].

D. Limitation and Outlook

As an endeavor in this emerging area, our current approach still has some limitations. The outpainting resolution in the sampling stage can be sensitive to the multi-scale training settings (e.g., when the resolution in the sampling stage is much larger than the maximal resolution of the target view in the training stage, EasyOutPainter seems to generate images with low quality, see Fig. 10). Another limitation is EasyOutPainter is an image-only-conditioned method, which does not support other modality controls. Besides, compared with other large-scale text-driven inpainting models [37], EasyOutPainter is more flexible, which can outpaint images with continuous multiple and resolutions. However, EasyOutPainter is trained on much less data (about 65,000 training images in total) and parameters (We use ViT-L as our U-ViT model, and the number of parameters is similar to previous SOTA QueryOTR [13], which is much smaller than SDXL [66]). Therefore, the generalization ability can be limited yet on the other hand, it enjoys the potential for deploying in end-side equipment like cell phones. We leave it for future work for how to incorporate specific prompts into our outpainting task, in accordance with GPT-like multi-modality models, which in fact is heavier than our current lightweight version.

V. CONCLUSION

We have proposed EasyOutPainter, which learns the positional relationships and pixel information at the same time. Methodically, EasyOutPainter can outpaint at any multiple and any resolution in only one step, greatly increasing the

applicability of image outpainting. Besides, since we propose a contrastive objective on the target view and anchor view, which enables EasyOutPainter training from scratch without extra pertaining cost (In contrast, previous methods are usually heavily influenced by the pre-trained encoder). We conduct experiments on three standard outpainting datasets, where EasyOutPainter achieves new SOTA results that surpass previous methods by a large margin under almost all settings. Depart from methodology, we also propose a more reasonable metric in outpainting areas (Center PSNR). Finally, we also conduct comprehensive ablation studies to show the robustness of our approach, including using classifier-free guidance, contrastive objective, multi-scale training, crop ratios, and the type of relative positional embeddings.

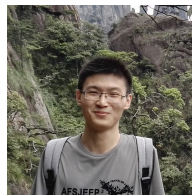
REFERENCES

- [1] C. H. Lin, H.-Y. Lee, Y.-C. Cheng, S. Tulyakov, and M.-H. Yang, “Infinitygan: Towards infinite-pixel image synthesis,” in *ICLR*, 2021.
- [2] Y.-C. Cheng, C. H. Lin, H.-Y. Lee, J. Ren, S. Tulyakov, and M.-H. Yang, “Inout: Diverse image outpainting via gan inversion,” in *CVPR*, 2022.
- [3] Y. Wang, Y. Wei, X. Qian, L. Zhu, and Y. Yang, “Sketch-guided scenery image outpainting,” *IEEE Transactions on Image Processing*, 2021.
- [4] W. Wang, J. Bao, W. Zhou, D. Chen, D. Chen, L. Yuan, and H. Li, “Sindiffusion: Learning a diffusion model from a single natural image,” *arXiv preprint arXiv:2211.12445*, 2022.
- [5] K. Kim, Y. Yun, K.-W. Kang, K. Kong, S. Lee, and S.-J. Kang, “Painting outside as inside: Edge guided image outpainting via bidirectional rearrangement with progressive step learning,” in *WACV*, 2021.
- [6] L. Zhang, J. Wang, and J. Shi, “Multimodal image outpainting with regularized normalized diversification,” in *WACV*, 2020.
- [7] Z. Yang, J. Dong, P. Liu, Y. Yang, and S. Yan, “Very long natural scenery image prediction by outpainting,” in *ICCV*, 2019.
- [8] X. Liu, M. Liu, J. Li, S. Liu, X. Wang, L. Lei, and W. Zuo, “Beyond image borders: Learning feature extrapolation for unbounded image composition,” in *CVPR*, 2023.
- [9] L. Dehan, W. Van Ranst, P. Vandewalle, and T. Goedemé, “Complete and temporally consistent video outpainting,” in *CVPR*, 2022.
- [10] M. Bertalmio, G. Sapiro, V. Caselles, and C. Ballester, “Image inpainting,” in *SIGGRAPH*, 2000.
- [11] F. Luan, S. Paris, E. Shechtman, and K. Bala, “Deep photo style transfer,” in *CVPR*, 2017.
- [12] C.-A. Yang, C.-Y. Tan, W.-C. Fan, C.-F. Yang, M.-L. Wu, and Y.-C. F. Wang, “Scene graph expansion for semantics-guided image outpainting,” in *CVPR*, 2022.
- [13] K. Yao, P. Gao, X. Yang, J. Sun, R. Zhang, and K. Huang, “Outpainting by queries,” in *ECCV*, 2022.
- [14] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, “Generative adversarial nets,” *NeurIPS*, 2014.
- [15] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *CVPR*, 2016.
- [16] B. Van Hoorick, “Image outpainting and harmonization using generative adversarial networks,” *arXiv preprint arXiv:1912.10960*, 2019.

- [17] P. Gao, X. Yang, R. Zhang, J. Y. Goulermas, Y. Geng, Y. Yan, and K. Huang, “Generalized image outpainting with u-transformer,” *Neural Networks*, 2023.
- [18] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, “Swin transformer: Hierarchical vision transformer using shifted windows,” in *ICCV*, 2021.
- [19] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, “Masked autoencoders are scalable vision learners,” in *CVPR*, 2022.
- [20] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly *et al.*, “An image is worth 16x16 words: Transformers for image recognition at scale,” in *ICLR*, 2021.
- [21] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” *NeurIPS*, pp. 6840–6851, 2020.
- [22] S. Zhang, J. Huang, Q. Zhou, Z. Wang, F. Wang, J. Luo, and J. Yan, “Continuous-multiple image outpainting in one-step via positional query and a diffusion-based approach,” in *ICLR*, 2024.
- [23] Z. Xie, Z. Zhang, Y. Cao, Y. Lin, J. Bao, Z. Yao, Q. Dai, and H. Hu, “Simmm: A simple framework for masked image modeling,” in *CVPR*, 2022.
- [24] J. Adler and S. Lunz, “Banach wasserstein gan,” *NeurIPS*, 2018.
- [25] A. Odena, C. Olah, and J. Shlens, “Conditional image synthesis with auxiliary classifier gans,” in *ICML*, 2017.
- [26] M. Kang, W. Shim, M. Cho, and J. Park, “Rebooting acgan: Auxiliary classifier gans with stable training,” *NeurIPS*, 2021.
- [27] D. Guo, H. Liu, H. Zhao, Y. Cheng, Q. Song, Z. Gu, H. Zheng, and B. Zheng, “Spiral generative network for image extrapolation,” in *ECCV*, 2020.
- [28] M. Arjovsky, S. Chintala, and L. Bottou, “Wasserstein generative adversarial networks,” in *ICML*, 2017.
- [29] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. C. Courville, “Improved training of wasserstein gans,” *NeurIPS*, 2017.
- [30] J. Kim, S.-H. Jeong, K. Kong, and S.-J. Kang, “An unified framework for language guided image completion,” in *WACV*, 2023.
- [31] M. Sabini and G. Rusak, “Painting outside the box: Image outpainting with gans,” *arXiv preprint arXiv:1808.08483*, 2018.
- [32] Y. Wang, X. Tao, X. Shen, and J. Jia, “Wide-context semantic image extrapolation,” in *CVPR*, 2019.
- [33] H. Lin, M. Pagnucco, and Y. Song, “Edge guided progressively generative image outpainting,” in *CVPR*, 2021.
- [34] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [35] C.-N. Lu, Y.-C. Chang, and W.-C. Chiu, “Bridging the visual gap: Wide-range image blending,” in *CVPR*, 2021.
- [36] F. Bao, C. Li, J. Zhu, and B. Zhang, “Analytic-dpm: an analytic estimate of the optimal reverse variance in diffusion probabilistic models,” in *ICLR*, 2022.
- [37] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” in *CVPR*, 2022.
- [38] F. Bao, C. Li, Y. Cao, and J. Zhu, “All are worth words: a vit backbone for score-based diffusion models,” in *CVPR*, 2023.
- [39] O. Ronneberger, P. Fischer, and T. Brox, “U-net: Convolutional networks for biomedical image segmentation,” in *MICCAI*, 2015.
- [40] S. Chen, P. Sun, Y. Song, and P. Luo, “Diffusiondet: Diffusion model for object detection,” *arXiv preprint arXiv:2211.09788*, 2022.
- [41] J. Ho, T. Salimans, A. A. Gritsenko, W. Chan, M. Norouzi, and D. J. Fleet, “Video diffusion models,” in *NeurIPS*, 2022.
- [42] X. Chen, Z. Zhao, F. Yu, Y. Zhang, and M. Duan, “Conditional diffusion for interactive segmentation,” in *ICCV*, 2021.
- [43] R. D. Hjelm, A. Fedorov, S. Lavoie-Marchildon, K. Grewal, P. Bachman, A. Trischler, and Y. Bengio, “Learning deep representations by mutual information estimation and maximization,” *arXiv preprint arXiv:1808.06670*, 2018.
- [44] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, “Momentum contrast for unsupervised visual representation learning,” in *CVPR*, 2020.
- [45] X. Chen, H. Fan, R. Girshick, and K. He, “Improved baselines with momentum contrastive learning,” *arXiv preprint arXiv:2003.04297*, 2020.
- [46] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, “A simple framework for contrastive learning of visual representations,” in *ICML*, 2020, pp. 1597–1607.
- [47] J.-B. Grill, F. Strub, F. Altché, C. Tallec, P. Richemond, E. Buchatskaya, C. Doersch, B. Avila Pires, Z. Guo, M. Gheshlaghi Azar *et al.*, “Bootstrap your own latent: a new approach to self-supervised learning,” *NeurIPS*, 2020.
- [48] X. Chen and K. He, “Exploring simple siamese representation learning,” in *CVPR*, 2021.
- [49] J. Zbontar, L. Jing, I. Misra, Y. LeCun, and S. Deny, “Barlow twins: Self-supervised learning via redundancy reduction,” in *ICML*, 2021.
- [50] S. Zhang, F. Zhu, J. Yan, R. Zhao, and X. Yang, “Zero-cl: Instance and feature decorrelation for negative-free symmetric contrastive learning,” in *ICLR*, 2021.
- [51] S. Zhang, L. Qiu, F. Zhu, J. Yan, H. Zhang, R. Zhao, H. Li, and X. Yang, “Align representations with base: A new approach to self-supervised learning,” in *CVPR*, 2022.
- [52] A. Bardes, J. Ponce, and Y. LeCun, “Vicreg: Variance-invariance-covariance regularization for self-supervised learning,” in *ICLR*, 2022.
- [53] X. Chen, S. Xie, and K. He, “An empirical study of training self-supervised vision transformers,” in *ICCV*, 2021.
- [54] M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, and A. Joulin, “Emerging properties in self-supervised vision transformers,” in *ICCV*, 2021.
- [55] J. Zhou, C. Wei, H. Wang, W. Shen, C. Xie, A. Yuille, and T. Kong, “Image BERT pre-training with online tokenizer,” in *ICLR*, 2022.
- [56] A. Van Den Oord, O. Vinyals *et al.*, “Neural discrete representation learning,” *NeurIPS*, 2017.
- [57] J. Ho and T. Salimans, “Classifier-free diffusion guidance,” in *NeurIPS 2021 Workshop on Deep Generative Models and Downstream Applications*, 2021.
- [58] W. R. Tan, C. S. Chan, H. E. Aguirre, and K. Tanaka, “Ceci n’est pas une pipe: A deep convolutional network for fine-art paintings classification,” in *ICIP*, 2016.
- [59] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, “Pytorch: An imperative style, high-performance deep learning library,” *NeurIPS*, 2019.
- [60] T.-C. Wang, M.-Y. Liu, J.-Y. Zhu, A. Tao, J. Kautz, and B. Catanzaro, “High-resolution image synthesis and semantic manipulation with conditional gans,” in *CVPR*, 2018.
- [61] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *ICLR*, 2019.
- [62] T. Salimans, I. Goodfellow, W. Zaremba, V. Cheung, A. Radford, and X. Chen, “Improved techniques for training gans,” *NeurIPS*, 2016.
- [63] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, “Gans trained by a two time-scale update rule converge to a local nash equilibrium,” *NeurIPS*, 2017.
- [64] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” *arXiv preprint arXiv:1409.1556*, 2014.
- [65] X. Liu, C. Gong *et al.*, “Flow straight and fast: Learning to generate and transfer data with rectified flow,” in *ICLR*, 2023.
- [66] D. Podell, Z. English, K. Lacey, A. Blattmann, T. Dockhorn, J. Müller, J. Penna, and R. Rombach, “Sdxl: Improving latent diffusion models for high-resolution image synthesis,” *arXiv preprint arXiv:2307.01952*, 2023.



Shaofeng Zhang received the Ph.D. degree from Shanghai Jiao Tong University in 2025. Before that, he received the BSc degree from the University of Electronic Science and Technology of China, Chengdu, China in 2021. He has published over 10 first-authored papers in ECCV, CVPR, SIGKDD, NeurIPS, ICLR, ICML on machine learning and computer vision, especially self-supervised learning.



Qiang Zhou obtained a PhD from Zhejiang University in 2019. He has been engaged in algorithm research and development at Alibaba Damo Academy and Inf Tech Company, and published several papers at CVPR, AAAI, ICLR.



Zhibin Wang received the Ph.D. degree from the University of Chinese Academy of Sciences, Beijing, China in 2015. He is currently the Staff Algorithm Engineer with Inf Tech Company, China. Before that, he was the Staff Algorithm Expert with the DAMO Academy, Alibaba Group. His research interests include deep learning, computer vision, and multimodal LLM.



Hao Li received the Ph.D. degree from the Chinese Academy of Sciences. He is currently a Professor with Fudan University. He has published more than 60 papers and owns more than 30 licensed patents. His research interests include multimodal large language models, AI for atmospheric science, deep learning model compression, and image search.



Junchi Yan (S'10-M'11-SM'21) is a full Professor with School of Artificial Intelligence, Shanghai Jiao Tong University (SJTU), Shanghai, China. Before that, he was a full-time (Senior) Research Staff Member with IBM Research from 2011 to 2018, and a consultant Senior Researcher with AWS AI Lab during 2019 to 2022. His research interests include machine learning and optimization. He is the Associate Editor for IEEE TPAMI, Pattern Recognition and the founding senior AE for ACM Trans. on Probabilistic Machine Learning. He also served regularly as Area Chair for ICLR, ICML, NeurIPS, CVPR etc. He is a Fellow of IAPR, AAIA, IET, and the recipient of the IEEE-CS AI'10 to Watch award and CVPR 2024 best paper candidate award. His five graduated PhD students have now been tenure-track faculties with Fudan University and SJTU.